```
object Worker {

  sealed trait Command
  final case class Parse(replyTo: ActorRef[Worker.Response], text: String)
      extends Command                                                    ❶

  sealed trait Response
  final case class Done(text: String) extends Response                   ❷

  ...
}
```

❶ Command that the actor can receive

❷ Response that the actor can send

The adapter function, once created, is used directly. It is passed in the `replyTo` of the `Parse` message along with the data to be parsed:

```
object Manager {
  ...
  worker ! Worker.Parse(adapter, text)
  ...
}
```

Finally, the worker's response is translated by the intermediate adapter function into the adapter message. The manager receives the `Worker.Done` wrapped in the adapter message and looks inside it to print the parsed text to the console. The manager code in the following listing shows how its four parts come together (message adapter, adapter function, delegating, and receiving).

Listing 3.2 Manager behavior

```
object Manager {

  sealed trait Command
  final case class Delegate(texts: List[String]) extends Command        ❶
  private case class WorkerDoneAdapter(response: Worker.Response)        ❶
```